RAG Knowledge Base: Smart Contract & Legal Agreement Audit Clauses

1. Introduction & Purpose

This document serves as the primary foundational knowledge base for the Retrieval-Augmented Generation (RAG) security audit model.

How the RAG Model Uses This Document:

1. **Embedding & Vectorization:** The clauses, code snippets, and explanations listed here are vectorized into the RAG's database.
2. **Similarity Search:** When a user inputs a smart contract (Solidity/Vyper code) or a legal Terms of Service (ToS) agreement, the model chunks the input and performs a semantic similarity search against this vector space.
3. **Classification:** Matched text is classified into Critical Risk, Moderate Risk, or Best Practice categories based on its proximity to the anchor clauses below.
4. **Contextual Generation:** The LLM generates the final audit report by directly retrieving the explicit explanations, vulnerability vectors, and legal risks mapped to the matched clauses, minimizing hallucinations and inference.

2. Critical Risk Clauses (High Severity)

High-risk clauses indicating potential fraud, critical vulnerabilities, or severe user disadvantage. The presence of these clauses indicates immediate danger to user funds or legal rights.

Category 2.1: Honeypot Patterns (Inability to Sell/Transfer)

1. **Selective Trading Lock:** `require(msg.sender == owner || isTradingEnabled);` *Explanation:* Prevents standard users from selling or transferring tokens while the owner can freely trade. Standard honeypot trigger.
2. **Hidden Blacklists:** `require(!blacklists[msg.sender]);` (Without clear governance limits) *Explanation:* Allows the deployer to freeze individual user funds arbitrarily, effectively trapping their capital.
3. **Dynamic Tax Trap:** `require(sellTax <= 99%);` or `function setTax(uint256 _tax) onlyOwner` *Explanation:* The owner can increase the sell tax to 99% or 100% post-launch, taking all proceeds from a user's sale.
4. **Max Transaction Throttling:** `require(amount <= maxTxAmount);` where `maxTxAmount` can be set to 0. *Explanation:* Halts all trading by setting the maximum allowed transfer amount to zero.
5. **Cooldown Manipulation:** `require(block.timestamp >= lastTx[msg.sender] + cooldownTimer);` *Explanation:* The owner can set the `cooldownTimer` to an impossibly high number (e.g., 100 years), soft-locking the tokens.

6. **Max Wallet Limits:** `require(balanceOf(to) + amount <= maxWalletSize);` (Where `maxWalletSize` is dynamically adjustable). *Explanation:* Can be manipulated by the owner to prevent users from receiving tokens or executing trades, acting as a stealth freeze.
7. **Approval Stripping:** `function approve(address spender, uint256 amount) public override returns (bool) { require(msg.sender == owner); ... }` *Explanation:* Disables the standard ERC20 approval mechanism for non-owners, preventing users from interacting with DEXs to sell their tokens.
8. **Fee-on-Transfer Traps:** Modifying balance mapping during `transfer` without updating standard events or variables properly. *Explanation:* Breaks standard DeFi composability and silently drains user balances during standard token movements.
9. **Fake Renouncement:** `require(msg.sender == address(0xDead) || tx.origin == originalOwner);` *Explanation:* The contract appears renounced (owner is a dead address), but the original deployer retains control through `tx.origin` checks.

Category 2.2: Rug Pull Patterns (Fund/Token Draining)

10. **Unlimited Minting:** `function mint(address to, uint256 amount) public onlyOwner` *Explanation:* Allows the owner to print infinite tokens, immediately diluting existing holders and draining liquidity pools.
11. **Treasury Drain / Emergency Withdraw:** `function emergencyWithdraw(address token) public onlyOwner` *Explanation:* If this applies to user-deposited staking funds (and not just protocol fees), it allows the developer to steal all Total Value Locked (TVL).
12. **Malicious Proxy Upgrades:** `function upgradeTo(address newImpl) public onlyOwner` (Without Timelock) *Explanation:* The owner can swap out a safe smart contract for a malicious one instantly, bypassing previous audits.
13. **Liquidity Unlocking:** `function unlockLiquidity() public onlyOwner` (Before promised lock date) *Explanation:* Allows developers to pull underlying assets (ETH/USDC) out of the decentralized exchange pool, crashing the price to zero.
14. **Hidden Owner Privileges:** `function setFeeReceiver(address _receiver) internal` triggered by obscure fallback functions. *Explanation:* Bypasses standard `onlyOwner` checks to redirect funds to a hidden developer wallet.
15. **Malicious Burn (Others' Tokens):** `function burnFrom(address account, uint256 amount) public onlyOwner` *Explanation:* Allows the deployer to arbitrarily destroy tokens belonging to other users, manipulating supply and seizing value.
16. **Arbitrary Delegatecall:** `function execute(address target, bytes memory data) public onlyOwner { target.delegatecall(data); }` *Explanation:* Allows the owner to execute malicious external code within the context of the current contract, easily destroying it or draining funds.
17. **Self-Destruct / Suicide:** `selfdestruct(payable(owner));` *Explanation:* Irreversibly deletes the smart contract and forwards all remaining native tokens (ETH) to the owner.

Category 2.3: Re-entrancy & Logic Vulnerabilities

18. **State Update Post-Transfer:** `(bool success, ) = msg.sender.call{value: amount}("");` `balance[msg.sender] -= amount;` *Explanation:* The classic Re-entrancy flaw. The external call executes before the user's balance is reduced, allowing malicious contracts to drain funds recursively.
19. **Missing Re-entrancy Guards on Withdrawals:** Withdraw functions lacking `nonReentrant` modifiers from OpenZeppelin. *Explanation:* Leaves the contract exposed to basic cross-contract state manipulation.
20. **Cross-Function Re-entrancy:** Two separate functions share state, but only one is protected by a re-entrancy guard. *Explanation:* Attackers can re-enter the unprotected function while the protected one is suspended in execution.
21. **Read-Only Re-entrancy:** `function getPrice() public view returns (uint256)` pulling from manipulated AMM spot prices during execution. *Explanation:* Allows attackers to trick external protocols relying on this contract's view functions into giving incorrect asset valuations.
22. **ERC777/ERC1155 Callback Exploits:** Processing `onERC1155Received` callbacks without locking the contract state. *Explanation:* Exploits the token standard's native callback functionality to hijack control flow.
23. **Flash Loan Price Manipulation:** Using `balanceOf(address(this))` or AMM spot prices to calculate rewards/shares. *Explanation:* Attackers can use flash loans to temporarily inflate the balance, tricking the contract into minting excessive rewards.
24. **Unchecked Return Values:** `token.transfer(msg.sender, amount);` (Without `require` or `SafeERC20`). *Explanation:* Some tokens do not revert on failure; they return `false`. Failing to check this allows state changes to occur even if the transfer failed.
25. **Signature Replay Attacks:** `require(ecrecover(hash, v, r, s) == owner);` (Without a nonce or chain ID check). *Explanation:* Attackers can reuse a valid signature from a past transaction or a different blockchain network to repeatedly execute an action.
26. **Phishing via tx.origin:** `require(tx.origin == owner);` *Explanation:* If the owner interacts with a malicious contract, that contract can call this function and bypass the authorization check, as `tx.origin` remains the owner.

Category 2.4: Hidden Fee Clauses & Economics

27. **Infinite Slippage Exploitation:** Code that ignores `amountOutMin` parameters and executes trades at high slippage to pocket the arbitrage. *Explanation:* Quietly steals value during swaps by front-running or sandwiching the user's own transaction.
28. **Dynamic Exit Fees:** "Exit fees are calculated based on current network TVL and may reach up to 50% during high volatility." *Explanation:* Traps user funds during market crashes when they need liquidity the most.
29. **Uncapped Management Fees:** "The Protocol deducts a daily management fee from staked assets." (Without a hardcoded cap). *Explanation:* A slow-drain rug pull. The protocol

can change the daily fee to 10% and drain accounts in weeks.

30. **Forced Conversion Fees:** Staking rewards are automatically converted to an illiquid native token at a mathematically disadvantageous exchange rate. *Explanation:* Extracts value by forcing users to absorb bad spread rates.
31. **Yield Skimming:** `uint256 actualYield = totalYield - (totalYield * undocumentedVariable / 100);` *Explanation:* Skims a percentage of generated yield before it is credited to the user, hidden deep in math libraries.
32. **Decimals Manipulation:** Obfuscating fee calculations by misaligning ERC20 token decimals (e.g., assuming 18 decimals for USDC which has 6). *Explanation:* Leads to massive over-charging of fees or miscalculation of user balances.

Category 2.5: Predatory Terms of Service (ToS) Clauses

33. **Arbitrary Account Freezing:** "The Company reserves the right to suspend, freeze, or terminate your account at any time, for any reason, without prior notice." *Explanation:* Violates core Web3 tenets of self-custody and provides legal cover for asset seizure.
34. **Total Liability Waiver (Exploits):** "The Protocol assumes zero liability for losses incurred due to smart contract bugs, hacks, or employee misconduct." *Explanation:* Attempts to absolve developers of gross negligence or insider rug pulls.
35. **Class-Action Waiver:** "You agree to waive any right to participate in a class-action lawsuit or class-wide arbitration against the Protocol." *Explanation:* Predatory legal shield preventing users from pooling resources to sue malicious developers.
36. **Unnotified Terms Modification:** "These terms may be updated at any time. Your continued use of the protocol constitutes acceptance of the new terms." *Explanation:* Allows the protocol to retroactively insert predatory clauses without giving users a chance to exit.
37. **Opaque Dispute Resolution:** "All disputes must be settled via binding closed-door arbitration in

$$High - RiskOffshoreJurisdiction$$

." *Explanation:* Makes legal recourse financially impossible for the average user.

38. **Immediate Asset Seizure Rights:** "In the event of a TOS violation, the Protocol reserves the right to confiscate all digital assets held within the user's account." *Explanation:* Direct theft clause hidden in legal jargon.
39. **Forced NDA on Exploits:** "Users agree not to publicly disclose any vulnerabilities, losses, or platform failures without prior written consent." *Explanation:* A gag order designed to prevent users from warning the community during an active rug pull or hack.
40. **Intellectual Property Surrender:** "By interacting with this smart contract, you surrender all intellectual property rights to data, strategies, and content generated." *Explanation:*

Highly predatory clause attempting to steal user-generated alpha or automated trading strategies.

3. Moderate Risk Clauses (Context-Dependent)

Clauses that are standard in the industry but introduce centralization vectors, moderate risk, or require specific operational context to determine their true safety.

Category 3.1: Liability & Legal Mechanics

41. **"As-Is" Provision:** "The software is provided 'as is', without warranty of any kind, express or implied." *Context Required:* Standard open-source MIT/GPL license text. Acceptable for decentralized code, but risky if the protocol holds custodial assets.
42. **Capped Damages:** "Liability shall not exceed the total fees paid by the user in the 12 months preceding the claim." *Context Required:* Standard SaaS clause, but highly inadequate for DeFi where users deposit millions in principal.
43. **Delaware/Singapore Governing Law:** "This agreement shall be governed by the laws of the State of Delaware / Singapore." *Context Required:* Standard tech hubs. Neutral, but excludes users from localized consumer protection laws.
44. **Arbitration in Recognized Hubs:** "Disputes shall be resolved via arbitration in London (LCIA) or Switzerland (SIA)." *Context Required:* Better than opaque offshore hubs, but still expensive for retail users.

Category 3.2: Privacy, Data, & Compliance

45. **Off-Chain Regulatory Screening:** "We may use third-party RPC providers to screen wallet addresses against OFAC sanctions lists." *Context Required:* Legally required in many jurisdictions, but compromises the censorship-resistance of the blockchain.
46. **Analytics Tracking:** "On-chain and off-chain behavioral data may be shared with analytical partners." *Context Required:* Acceptable if purely statistical; dangerous if linked to personal identifiers (KYC/IP data).
47. **Mandatory KYC/AML:** "Users must complete identity verification before withdrawing funds exceeding $10,000." *Context Required:* Standard for centralized exchanges (CEX), highly unusual and centralizing for decentralized finance (DeFi).
48. **Geoblocking Execution:** Code or ToS that restricts access based on IP address (e.g., blocking US or EU residents). *Context Required:* Legally protective for the developers, but introduces a centralized point of failure at the frontend level.

Category 3.3: Operations & Centralization Vectors

49. **Notice of Termination:** "The Protocol may sunset its frontend interface with 30 days prior written notice." *Context Required:* 30 days is standard, but the RAG must verify if a decentralized smart-contract withdrawal fallback exists.
50. **Manual Unwinding:** "Upon account termination, users are responsible for manually unwinding their DeFi positions." *Context Required:* Reasonable technically, but risks users

losing funds to liquidations if the UI is taken down prematurely.

51. **Pausability (Circuit Breakers):** `import "@openzeppelin/contracts/security/Pausable.sol";` *Context Required:* Great for stopping hacks in progress, but acts as a temporary honeypot if the developer goes rogue. Must be controlled by a multi-sig.
52. **Proxy Implementation:** `contract TransparentUpgradeableProxy` *Context Required:* Allows bug fixes and logic upgrades. Introduces severe rug-pull vectors if not combined with a Multi-Sig and a Timelock.
53. **Single Admin Role:** `modifier onlyAdmin()` applied to system-critical variables. *Context Required:* Standard for early-stage startups, but represents a massive centralization risk if the admin key is compromised.
54. **Oracle Dependencies:** Reliance on a single external price feed (e.g., a single Chainlink oracle). *Context Required:* High quality, but if the specific token pair lacks liquidity, the oracle can be manipulated.
55. **Dynamic Gas Fees:** "The protocol will deduct estimated network gas costs from the final withdrawal amount." *Context Required:* Standard UX practice, but the smart contract must hard-cap how much it can overestimate gas.
56. **Variable Swap Fees:** "Swap fees range from 0.01% to 1.5% depending on pool volatility." *Context Required:* Acceptable automated market maker (AMM) mechanics, provided the maximum bounds are hardcoded in Solidity.

4. Best Practice Clauses (Low Risk)

Clauses indicating a strong security posture, user-centric design, code transparency, and robust risk mitigation. These reduce the overall risk score.

Category 4.1: Smart Contract Safety & Architecture

57. **Re-entrancy Guards:** `modifier nonReentrant()` applied to all state-changing external functions. *Explanation:* Demonstrates adherence to basic security hygiene against cross-function and external re-entrancy attacks.
58. **Zero-Address Validation:** `require(account != address(0), "Cannot transfer to zero address");` *Explanation:* Prevents accidental burning of tokens or assigning critical administrative roles to null addresses.
59. **Safe ERC20 Usage:** `using SafeERC20 for IERC20;` and using `.safeTransfer()` instead of `.transfer()`. *Explanation:* Ensures compatibility with non-standard tokens (like USDT) that don't return boolean values on failure.
60. **Hardcoded Timelocks:** `require(block.timestamp >= scheduledExecutionTime, "Timelock active");` *Explanation:* Gives users a mandatory grace period (e.g., 48 hours) to withdraw funds before a developer can execute a critical contract upgrade.
61. **Check-Effects-Interactions Pattern:** State variables are updated *before* making any external calls. *Explanation:* The fundamental architectural pattern for preventing re-

entrancy, independent of specific modifiers.

62. **TWAP Oracles:** Use of Time-Weighted Average Price (TWAP) instead of spot prices. *Explanation:* Mathematically mitigates flash loan price manipulation by averaging asset prices over a specific time block.
63. **Role-Based Access Control (RBAC):** `import "@openzeppelin/contracts/access/AccessControl.sol";` *Explanation:* Distributes permissions securely instead of relying on a single `onlyOwner` modifier.
64. **EIP-712 Structured Signatures:** Implementation of standard domain separators and type hashes for off-chain signing. *Explanation:* Prevents signature replay attacks across different contracts and different blockchain networks.
65. **Explicit Role Revocation:** `function renounceRole(bytes32 role, address account) public virtual;` *Explanation:* Allows administrators to permanently step down and decentralize the protocol over time.

Category 4.2: User Rights & Data Protection

66. **Absolute Self-Custody Guarantee:** "Users retain full, unencumbered self-custody of their digital assets. The Protocol cannot access or freeze user funds." *Explanation:* The gold standard for DeFi legal agreements, aligning legal text with smart contract reality.
67. **Sufficient Notice Periods:** "Any material changes to these Terms will be communicated via official channels at least 60 days prior to taking effect." *Explanation:* Allows users ample time to evaluate changes and exit the system if desired.
68. **Immutable UI/Fallback:** "The smart contracts are entirely immutable and can be interacted with directly via Etherscan or IPFS fallback interfaces indefinitely." *Explanation:* Guarantees users can always access their funds even if the company goes bankrupt or the central website goes offline.
69. **Opt-in Data Collection:** "No off-chain tracking (cookies, IP logging) is performed without explicit, opt-in consent from the user." *Explanation:* Highly respectful of Web3 privacy ethos and complies strictly with GDPR frameworks.

Category 4.3: Economics, Fees & Governance

70. **Hardcoded Fee Ceilings:** `require(newFee <= MAX_FEE_BPS, "Fee exceeds maximum");` (e.g., Max 2%). *Explanation:* Mathematically prevents the developer from extracting predatory fees in the future, overriding any vague ToS language.
71. **DAO Governance on Fees:** "All parameter changes, including fees, must be approved by a decentralized governance vote achieving a 51% quorum." *Explanation:* Removes unilateral economic control from the core development team.
72. **Zero Exit Fee Guarantee:** "No withdrawal, exit, or unstaking fees shall ever be applied to user principal deposits." *Explanation:* Protects the user's initial investment from hidden extraction.
73. **On-Chain Fee Auditing:** "All protocol fee calculations are fully transparent, documented at

URL

, and match the open-source smart contract logic." *Explanation:* Ensures that legal/documentation text strictly matches code execution.

Category 4.4: Fund Protection & Recourse Mechanisms

74. **Pull Over Push Pattern:** `function withdrawBalances() public { uint amount = balances[msg.sender]; balances[msg.sender] = 0; payable(msg.sender).transfer(amount); }` *Explanation:* Safest way to handle mass token distribution or withdrawals, completely preventing denial-of-service (DoS) attacks from failing external calls.
75. **Active Bug Bounty:** "The Protocol maintains an active $1,000,000 bug bounty program on Immunefi." *Explanation:* Incentivizes white-hat hackers to find and report bugs responsibly before malicious actors exploit them.
76. **Decentralized Multi-Sig Treasury:** "The protocol treasury and upgrade functions are controlled by a 5-of-7 multi-signature wallet comprising independent security researchers." *Explanation:* Eliminates the "single point of failure" risk of a single rogue developer or compromised private key.
77. **Protocol Insurance Coverage:** "Smart contract risks are covered up to $20M by decentralized insurance protocols (e.g., Nexus Mutual)." *Explanation:* Provides actual, tangible financial recourse for users in the event of an unforeseen exploit.
78. **Rate Limiting / Circuit Breakers:** `require(withdrawnToday[msg.sender] <= dailyLimit, "Daily withdrawal limit reached");` *Explanation:* In the event of an exploit, this mathematically limits the amount of capital a hacker can drain per day, giving the multi-sig time to pause the contract.

5. Pattern Matching Reference (Trigger Keywords)

This section provides RegEx and Keyword anchors for the RAG chunking algorithm to classify specific text blocks.

Risk Tier	Category	Keyword/RegEx Triggers for RAG Parsing
CRITICAL	Honeypot	`require\(msg\.sender == owner\)` , `blacklist` , `isTradingEnabled` , `cooldown` , `maxTx` , `tx\.origin ==`
CRITICAL	Rug Pull	`function mint\(.*\) onlyOwner` , `emergencyWithdraw` , `upgradeTo\(.*\)` , `selfdestruct` , `delegatecall`
CRITICAL	Re-entrancy	`\.call\{value:` , `\.transfer\(` *(prior to balance update)*, missing `nonReentrant` , `ecrecover`

CRITICAL	Predatory ToS	"freeze", "terminate at any time", "class-action waiver", "without notice", "surrender all intellectual property"
MODERATE	Liability/Privacy	"as is", "OFAC", "third-party analytics", "governing law", "arbitration", "KYC"
MODERATE	Smart Contracts	`Pausable` , `Proxy` , `onlyAdmin` , "estimated gas costs", "Chainlink"
BEST PRAC.	Safety/Custody	`ReentrancyGuard` , `SafeERC20` , `Timelock` , "self-custody", "immutable", `AccessControl`
BEST PRAC.	Protection	`MAX_FEE` , "multi-sig", "DAO governance", "bug bounty", "pull over push", "TWAP"

6. Risk Scoring Guide

The RAG model utilizes the following deterministic formula to calculate the final Trust Score of a contract/agreement.

Base Score: `100` (Perfectly Safe)

Deductions & Additions:

- **Critical Risk Clause Found:** `-15 points` per instance.
- **Moderate Risk Clause Found:** `-5 points` per instance.
- **Best Practice Clause Found:** `+5 points` per instance.

Critical Multipliers & Thresholds:

- **The "Do Not Interact" Threshold:** If the score drops below `40` , the asset is automatically flagged as **"HIGH RISK / DO NOT INTERACT"**.
- **Critical Vulnerability Override:** If any *Re-entrancy*, *Arbitrary Delegatecall*, or *Unlimited Minting* Critical clause is detected, the maximum possible score is capped at `30` , regardless of how many Best Practice clauses exist.
- **Pausable + Timelock Synergy:** If `Pausable` (Moderate) is detected, but a `Timelock` (Best Practice) is also present, the Moderate penalty is negated (`0 points` deducted instead of `-5`).

Formula: `Final Score = MIN(100, Base (100) - (Critical Count * 15) - (Moderate Count * 5) + (Best Practice Count * 5))` *(Subject to Override Caps)*

7. Sources & References

The following sources were used to compile these standard audit and legal clauses. RAG should cite these URLs when explaining risks to users.

1. **Smart Contract Weakness Classification (SWC Registry)** *URL:* https://swcregistry.io/
   Use Case: Re-entrancy, tx.origin, delegatecall, and standard smart contract vulnerability

definitions.

2. **OpenZeppelin Contracts Documentation** *URL:* https://docs.openzeppelin.com/contracts/ *Use Case:* Standards for SafeERC20, Pausable, AccessControl, and ReentrancyGuard.
3. **Consensys Diligence - Smart Contract Best Practices** *URL:* https://consensys.github.io/smart-contract-best-practices/ *Use Case:* Pull-over-push patterns, circuit breakers, and architectural safety.
4. **Immunefi - Vulnerability Severity Classification System** *URL:* https://immunefi.com/severity-system/ *Use Case:* Categorization of critical rug-pulls and fund drainage vectors.
5. **DeFi Safety - Protocol Rating Methodology** *URL:* https://www.defisafety.com/methodology *Use Case:* Multi-sig, timelock, and documentation best practices.
6. **LexDAO - Web3 Legal Engineering** *URL:* https://lexdao.coop/ *Use Case:* Standardized Web3-native Terms of Service and arbitration clauses.
7. **SEC - Framework for "Investment Contract" Analysis of Digital Assets** *URL:* https://www.sec.gov/corpfin/framework-investment-contract-analysis-digital-assets *Use Case:* Contextualizing liability, governance, and security classifications.